
Practical AI for Academics

Day 2 — hands-on with your own materials

Claes Bäckman

Leibniz Institute for Financial Research SAFE

Three messages: ground the agent in *your* files, run it as a critic before you publish, and keep verification close.

1 From mental model to your draft

Day 1 was tools and theory — **today we point them at your own paper, prose, and slides.**

Implication: the leverage is in materials only you have.

2 Critic, not co-author

Eight subagents reading one draft beat one diffuse pass. The skill is **cheap pre-flight checking**, not replacing the referee.

Implication: use AI to find what you would miss.

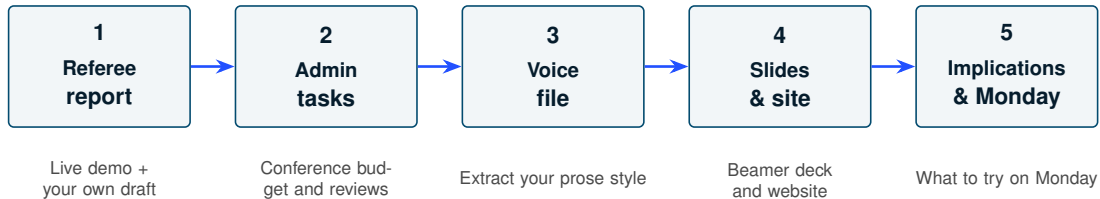
3 The bottleneck is judgement

Producing draft prose, slides, and websites is now cheap. **Reading, checking, and disagreeing** is where the researcher's edge sits.

Implication: verification is the skill to invest in.

Key takeaway. By the end of today you will have run a referee report on your own draft, built a `voice.md`, and seen slides and a website generated from your materials.

Day 2 agenda — five hands-on blocks, ending with implications.



What you walk out with. A referee report on a draft you actually care about, a `voice.md` from your own writing, and a list of three things to try next week.

Everything today assumes the Day 1 setup: VS Code, Claude Code, and a `CLAUDE.md` in the project folder.

Slides, skills, and guides for both days: claesbackman.com/aarhus

Setting up your workspace

Five steps get VS Code + Claude Code working in under five minutes.

1. Open VS Code and the Extensions panel (Ctrl+Shift+X / Cmd+Shift+X)
2. Search “**Claude Code**” and install *Claude Code for VS Code* (extension ID `anthropic.claude-code`)
3. Restart VS Code — Claude Code opens in the sidebar
4. Sign in with your Claude account (Pro/Max/Team/Enterprise all include Claude Code) or an API key
5. Open your **full project folder** (File > Open Folder) — never a single file

What an IDE actually is. Editor + file explorer + terminal + extensions, in one window. VS Code is the dominant free one. Think of it as Word or R Studio for code, LaTeX, markdown, and plain text.

A handful of extensions cover Stata, Python, LaTeX, and data

Stack	Extension ID	What it does
Stata	<code>tmonk.stata-workbench</code>	Run Stata from the editor
Python	<code>ms-python.python</code>	Syntax, IntelliSense, run button
LaTeX	<code>james-yu.latex-workshop</code>	Compile on save, side-by-side PDF preview, inline errors
Data	<code>mechatroner.rainbow-csv</code>	Colour-coded CSV columns; SQL-like filtering
	<code>tomoki1207.pdf</code>	Renders PDFs inline — read codebooks next to your code

Key takeaway. Install Python even if you do not write it. The agent uses Python as glue for many different tasks.

You can link Overleaf to your own computer and work in that folder

Why bother? Overleaf has no Claude Code, no Codex, no Copilot, no proper diff view, no terminal. The fix: write in VS Code and let Overleaf follow what you produce.

Requirement. Both routes need **Overleaf Premium** — many universities provide it through the institutional login.

Dropbox or Github?

	GitHub	Dropbox
Sync	Manual push and pull	Automatic, within seconds
History	Full git history, branches	Dropbox revisions only
Best for	Solo work, versioned drafts	Live coauthoring in Overleaf
Conflicts	Git merge	“Conflicted copy” files

Key takeaway. **Pick one per project.** Both syncs at once means duplicate-file chaos.

Two routes: GitHub Desktop for a paper trail (click to commit, no terminal), Dropbox for live coauthoring.

Setup A GitHub via GitHub Desktop

1. Overleaf: Menu → Integrations → GitHub → **Export Project to GitHub**
2. GitHub Desktop: File → Clone repository, pick the repo and a local folder
3. Open that folder in VS Code
4. Edit in VS Code. In GitHub Desktop: one-line summary, **Commit to main**, **Push origin**
5. Overleaf: Menu → GitHub → **Pull GitHub changes into Overleaf**
6. Coauthor edited in Overleaf? **Push Overleaf changes to GitHub**, then pull in GitHub Desktop

Accept the TeX .gitignore template: no .pdf or .aux in the history.

Setup B Dropbox

1. Install Dropbox and let the initial sync finish
2. Overleaf: Menu → Dropbox Sync → [Link](#). Overleaf creates Apps/Overleaf/ with one subfolder per project
3. Open the project folder in VS Code and edit as normal — every save reaches Overleaf within seconds, and Overleaf edits flow back
4. Compile in Overleaf as usual, or locally with `latexmk -pdf`

No commit step. Good for active coauthoring, less good if you want a paper trail.

Working with the agent

Every Claude Code session follows the same pattern — open the folder, set context, then prompt.

1. Open VS Code and the project folder (File → Open Folder)
2. Open the Claude Code panel in the sidebar — or the terminal (Ctrl+`) and type `claude`
3. Confirm Claude is pointed at the **full project folder** — not a single file
4. Send a first orienting prompt

First prompt to try:

“Read the files in this folder and tell me what this project is about.”

Expected behaviour. Claude reads the folder, summarises the structure, and asks you what you want to do next. **If it does not ask a question, your CLAUDE.md should tell it to.**

CLAUDE.md is the standing-instructions file — write it once, stop repeating yourself.

About me

I am a researcher in [field]. I write papers, lecture notes, policy pieces.

How I want you to work with me

- Ask clarifying questions before generating long output.
- Keep a skeptical, critical tone in feedback. Do not flatter.
- When editing, preserve my voice. Avoid generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality when summarising empirical work.
- Do not insert emoji or markdown decorations in formal documents.

A few files hold your durable project context — know what goes where.

File	Use it for
<code>project/CLAUDE.md</code>	Project instructions. Dataset structure, sample restrictions, paper title, coauthor conventions, which script produces which output.
<code>~/.claude/CLAUDE.md</code>	Personal preferences across every project: writing style, default tone, citation conventions. Both files load; the project file wins on conflict.
<code>voice.md</code>	Writing samples. Paragraphs, abstracts, and tone notes that Claude should match when editing prose.
<code>.claude/skills/...</code>	Reusable workflows. Named tasks you want to run again without rewriting the prompt.

Key takeaway. Keep secrets and raw confidential data **out of all of them** — these files are read into context on every session and may be sent to the model provider.

Bootstrap CLAUDE.md in two minutes — type `/init`.

The built-in command: type `/init` in the Claude Code panel. Or spell it out:

“Read this project folder and draft a CLAUDE.md file with the project structure, conventions, and instructions you should follow. Do not edit anything else yet.”

What Claude does

- Reads the folder
- Infers languages, structure, conventions
- Generates a CLAUDE.md draft

Then you edit it. Add tone preferences, things to avoid, sample restrictions Claude could not infer.

Plan mode — think before you write. The agent plans, you correct, then it executes.

For anything non-trivial, ask Claude to outline its approach **before** generating.

Example prompt:

“Before writing the policy brief, outline the structure, the three findings you plan to lead with, and the one caveat you’ll flag. Do not write the brief yet. Ask me questions if you need to.”

You review the plan, correct the framing, then approve it. Claude Code has an explicit **plan mode**:

- Accept the plan and it executes; reject it and you have lost thirty seconds

Key takeaway. Planning catches the wrong approach when it is cheap to fix — before the agent has written 800 lines you have to read.

Permissions follow a tiered model

Plan mode

Reads and proposes; cannot write.

Manual

Asks before each edit or command.

Edit automatically

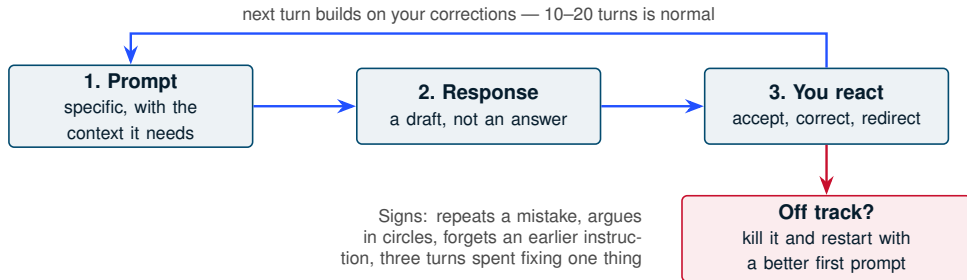
Claude will edit your selected text or the whole file

Auto mode (default mode now)

Claude will approve actions that pass a safety check and pause for anything risky

Claude Code should stay inside the folder you opened, but this is not certain. Use `/permissions` to see the current rules, and `allow-` or `block-list` specific commands (`Rscript`, `pdflatex`, `rm`) in `settings.json`.

A session is a conversation, not a query — iterate, steer, and kill it when it drifts.



Continue the conversation. The context carries your corrections, so later answers improve: *“Shorter, and use the Danish case.” “You misread Table 2 — look again.” “Now the same for Section 3.”*

Kill it when you veer off track. Sunk cost is zero. Start fresh and put what you learned into the first prompt — or into `CLAUDE.md` so you never repeat it. In Claude Code: `/clear`. In a chatbot: a new conversation.

Key takeaway. The restart is the most under-used move. A fresh session with a better first prompt beats twenty turns of repair.

Five prompting rules: specific, iterate, correct early, verify, restart.

1. **Specific.** “Feedback on my paper” is a worse request than “Give me feedback on the identification strategy in Section 4.”
2. **Iterate.** 10–20 turns, not one.
3. **Correct early.** If the first answer is off, fix it now — or restart the chat.
4. **Trust but verify.** Run the code. Re-read the passage. Check the citation.
5. **Open new sessions.** 30+ turns or the task has shifted — `/clear` and restart.

Key takeaway. The researcher who sends one long prompt and accepts the answer gets generic output. The researcher who treats the agent like a junior colleague and argues for ten turns gets work product.

Four failure modes to recognise — flattery, generic feedback, fabrication, drift.

Failure mode	Symptom	Fix
Flattery	<i>"This is a well-structured piece."</i>	Ignore. Ask for the strongest objection a hostile referee would raise.
Generic feedback	<i>"Consider strengthening the motivation."</i>	Push back: "Give me a specific sentence that would make it stronger."
Confident fabrication	Claude describes a passage that is not in the draft, or cites a paper that does not exist.	Always re-read the passage. Always check the citation.
Session drift	Repeats mistakes, invents files, gives circular answers, forgets the last three turns.	/clear and start fresh. Cost: 5 seconds.

A referee report on your paper

Pick one draft you would like another pair of eyes on. Preferably a relatively close-to-finished draft

How to get started

Get the skills from claesbackman.com/aarhus — download the zip, or paste the one-line terminal command on that page. Source and updates:
github.com/claesbackman/AI-research-feedback.

Unzip, then copy each skill folder (review-paper, review-paper-light, ...) into
~/.claude/skills/ — a hidden folder under your user folder (Windows:
C:\Users\you\.claude\skills)

- To show hidden folders: Open File Explorer, click the View tab at the top, select Show, and check Hidden items
- A skill in ~/.claude/skills/ works in every project; one in project/.claude/skills/ only in that project

Restart VS Code

The skill loads when you type /skill-name

Live demo — /review-paper on a real draft, eight subagents in parallel.

Setup

- One of my own papers, in its own folder
- .tex source, .bib, supporting materials
- One command: /review-paper with a target journal (AER)

Output

- A structured markdown report
- Referee-style comments, grouped by theme
- Specific line and section references

Eight subagents running in parallel

- Spelling, grammar & style
- Internal consistency & cross-references
- Unsupported claims & identification
- Mathematics, equations & notation
- Tables, figures & documentation
- Referee assessment: positioning & fit
- Contribution advocate
- Contribution skeptic

Pattern: eight narrow critics beat one diffuse one.

Key takeaway. The skill is a ~200-line markdown file. You can read it, edit it, and download it yourself.

Hands-on — run a referee report on your own writing.

1. Open one of your own drafts in VS Code
2. Make sure it is in markdown or LaTeX — convert if needed
3. Run `/review-paper-light` for a 1–2 minute version, or `/review-paper` for the full eight-agent run
4. Read the output. It's not all correct!

Debrief questions

- What did the report catch that you would not have seen by yourself?
- What did it get wrong?
- Which subagent gave you the most useful feedback?

A skill is a markdown file the agent loads on demand — invoke with `/name`.

A skill lives at `.claude/skills/your-skill/SKILL.md` (or `~/.claude/skills/` for every project):

`name: review-paper`

`description: Run an 8-agent pre-submission referee report for an academic paper targeting a specified journal`

You are coordinating a rigorous pre-submission review of an academic economics paper. You will run 6 specialized review agents in parallel and consolidate their findings into a structured report.

Invoke with `/review-paper`. Claude loads the instructions and runs them.

CLAUDE.md is always loaded, skills are on demand.

CLAUDE.md — always loaded

- Loaded into context every session
- Background, conventions, voice
- “How I want you to work with me”

Cost: *permanent context budget.*

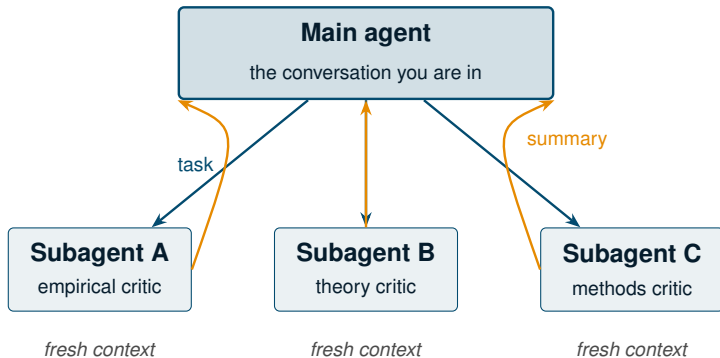
Skill — on demand

- Loaded only when invoked with /name
- A specific workflow, with structured output
- “Do this thing for me”

Cost: *zero until triggered.*

Key takeaway. If you find yourself pasting the same prompt multiple times, you can ask Claude to make it into a skill.

Subagents add two things the main agent cannot do alone — parallelism and context isolation.



Key takeaway. The /review-paper skill spawns **eight** subagents.

The second-opinion pattern — spawn an adversary to check on the work.

The main agent sees its own prior answer and has reason to defend it.

Why a fresh subagent is different

- No chat history: no anchoring on what you already wrote
- No conversational momentum: no reason to agree with the main agent
- Clean context: can hold the whole artifact at once

When to spawn one

- Before accepting a long answer: *“is this actually right?”*
- After a long refactor: *“what did we break?”*
- When you suspect flattery: *“give me the honest version”*
- Before sending: *“read this as a hostile reader”*
- After doing automatic edits: *“look over the changes we did”*

Key takeaway. The phrases “**spawn a subagent**” or “**launch an agent**” are the trigger words. The main agent knows what to do and will write the instructions for you.

What the demo is meant to show

1. **Grounded output.** The report is about *your* draft, not a generic checklist
2. **Subagents are why this works.** Eight narrow critics catch things one diffuse reader misses
3. **Not a replacement for a referee.** It is a pre-flight check before you submit
4. **Transparent.** The skill is a $\sim$ 200-line markdown file — fork it, modify it, make it yours

Key takeaway. The point is not to trust the report. The point is to make hard feedback cheap enough that you ask for it before submitting — not after the desk reject.

Other tools generate referee reports too

Tool	What it does	Pricing
Refine.ink	Referee-style report on a paper PDF	<i>Paid</i>
coarse.ink	Lightweight version of the same idea	<i>≈ free</i>
Referee 2	Paid referee-report tool	<i>Paid</i>
Stanford Agentic Reviewer	Open agentic reviewer	<i>Free</i>

Many more out there — new ones launch every few weeks.

Key takeaway. The advantage of running your own skill is that it lives next to your draft, knows your conventions, and you can edit what it checks.

Now launch another agent to critique the referee report you just generated — ask Claude to write the instructions.

Working with secure data

Secure servers split the workflow

What Claude Code can still help with on your laptop

- Your `.do` files
- Writing and iterating on empirical strategy

Claude cannot run the files or read the logs on the server, but can still be very helpful in a number of ways

Two artifacts to keep local

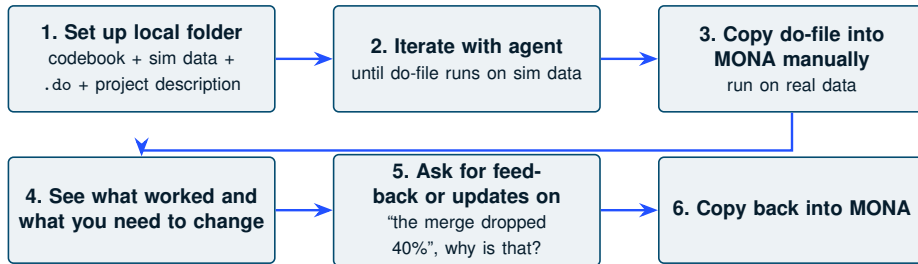
1 Codebook in markdown

- Convert once (`/pdf-to-markdown`)
- Variable names, labels, value ranges, missing-value codes
- Agent reads it before writing `recode`, `label define`, `merge`

2 Simulated dataset

- Same variable names, types, ranges — not real values
- AER/JF/ReStud replication packages often ship with these
- Or generate from the codebook

A workflow for secure data on servers



Key takeaway. The agent never sees the real data — but it sees everything that explains the data. That is usually enough to write correct Stata code.

Organising a conference

Live demo: Administrative tasks Claude Code can take off your desk.

Setup

- A folder with a call for papers

The goal

- Build a budget for the conference

The output

- A budget spreadsheet with line items, totals, and assumptions

What we are using. Plain Claude Code prompts — no skills, no MCP. Just the folder and the agent.

Two points the admin demo is meant to show

1. AI tools reduce the cost of administrative work.

- Not just for research
- Often the highest-leverage use case, because the friction is the highest and the quality bar is the lowest

2. Context is the difference between useful and useless.

- A chat tool gives you a generic conference-budget template
- Claude Code finds your keynote speakers, your dinner, and your PhD travel funding by reading the folder

Key takeaway. The same pattern — open the folder, hand the agent the materials, ask for the output — works for grant reports, course design, referee assignments, and committee memos.

Writing with agentic AI

What are the three phrases you hate seeing in AI-edited prose?

A `voice.md` file makes edits sound like you

When editing my prose, match the voice in `voice.md`.
Do not use em-dashes, bullet lists, or passive constructions unless the original does.
Do not use "it's not x, it's y"-style writing.

What goes in it

- 5–10k words of your prose — intros, abstracts, paragraphs you are proud of

Key takeaway. Reference it from `CLAUDE.md` to give Claude writing context for every session

Hands-on (15 min) — build a `voice.md` from your own writing.

We use a skill written by Mihail Velikov: `voice-extractor`.

1. Upload 4 PDFs of your own writing to a folder
2. Call the skill: `/voice-extractor sample-papers`
3. Read through `voice.md` and verify — edit anything that does not sound like you

What goes in. Intros, abstracts, paragraphs you are proud of. 5–10k words is plenty.

What stays out. Co-authored sections where the prose is not yours. AI-edited drafts.

Key takeaway. Reference `voice.md` from `CLAUDE.md`. Every future session loads your voice into context.

How to tell whether your `voice.md` is working

Signs it is working

- Edited paragraphs read more like you, less like a generic LLM
- Fewer em-dashes, fewer bullet lists, fewer “crucially” / “moreover”
- You stop having to say “preserve my voice” in every prompt
- Your coauthors stop noticing which paragraphs were AI-edited

Signs it is not

- Output still sounds generic
- AI tics persist after editing
- You keep adding “don’t use X” to every prompt

***Fix:** sample is too short, or includes too much AI-edited prose. Add more of your writing.*

Key takeaway. `voice.md` is a living file. Update it when your style shifts, or when an editor pushes you in a direction you want to keep.

AI detectors: treat the score as a signal, never as a verdict.

How they work. A classifier trained on the fingerprint of model text — predictable word choice, even rhythm, stock phrasing. It returns a probability, not evidence.

Why the first generation failed. False positives, concentrated on non-native writers and formulaic academic prose. OpenAI withdrew its own classifier in 2023; several universities switched off Turnitin's score.

Pangram. The strongest current detector, with very low reported false-positive rates. Still a probability, and it cannot tell you *how* AI was used: a student who used it to learn, then wrote the essay, looks like one who never touched it.

What to do instead

1. **Never accuse on a score alone.** Use it to decide whom to talk to, not what grade to give. Ask the student to explain their work.
2. **Set a per-assignment AI policy.** State plainly what is allowed where — ambiguity causes the problems, not the tool.
3. **Move the stakes to where understanding shows:** closed-book exams, in-class work, oral exams. Then using AI to learn pays off and avoiding learning does not — and detection matters much less.
4. **Teach the good use.** *“Act as a tutor: ask me questions and give hints, do not give the answer.”* Publish answer keys.

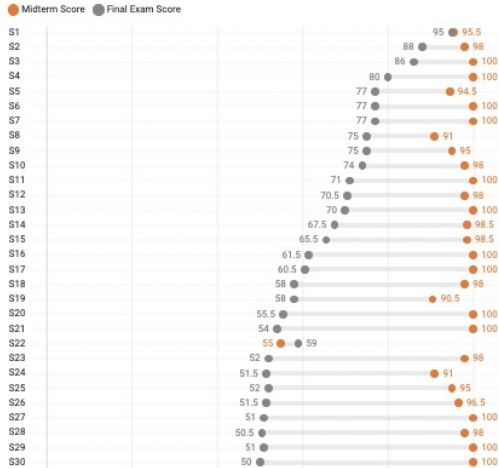
SECTION 7

Teaching

Go through your syllabus. Which take-home assignment worth non-trivial credit could a student complete with a frontier model today?

ECON 1170 Midterm & Final Scores

After Serrano made the final in-person, 18 students dropped the class and nine students did not take the exam. The scores of the remaining 59 students are displayed here.



AI breaks the grade–knowledge link, and the pre-AI classroom is no benchmark.

Three things AI changes, according to Bryan

1. **The grade–knowledge link is broken.** A good take-home grade no longer shows the student can do the thing.
2. **Students need to use AI well later.** Banning it everywhere leaves them untrained.
3. **AI should let us teach more.** Tutoring, feedback, and individual practice used to be too expensive.

Bryan (2026). All Day TA, used in several of his examples, is Bryan's own company.

The pre-AI status quo was already leaky

- US college study time fell from **40 to 27 hours a week**, 1961–2003
- A quarter of finance students copied **wrong** Chegg answers, and learned less

And AI makes homework a worse signal

- Post-AI: homework scores +18%, completion time –30%, exam performance –20% (China)
- Time on AI-susceptible homework collapsed. Exam performance on those topics fell 25%

Key takeaway. Modify the course, the expectations, and the evaluation, and education can come out **more** effective than before.

Eight rules, two halves: protect AI-free learning, then use AI to teach more.

Protect learning

1. **Decide what should be learned without AI.** Be explicit about which assignments test AI-independent competence.
2. **Teach better than the pre-AI status quo.** Going back a generation is not the goal.
3. **Give students an incentive to learn.** Regular, graded checks. Effort follows tests.
4. **Do not give AI-cheatable assignments.** Every take-home worth real credit can be done by a frontier model today.

Teach more with AI

5. **Help students learn more efficiently.** Class-specific tutoring, spaced repetition, mastery learning at scale.
6. **Use AI to improve your own teaching.** Find out where you were confusing before the exam tells you.
7. **Personalise assignments.** Problems at the level each student is struggling with.
8. **Raise standards.** If research and drafting are cheaper, the A-grade bar moves up.

Key takeaway. Rules 1–4 are about the **assessment** you already have. Rules 5–8 are about things you could not afford before. Start with the first half.

Rules 1 and 4: audit what each assignment certifies and whether AI can do it.

How to run the audit

1. Syllabus, all assignments, one past exam in a folder
2. Ask Claude Code for the audit (right)
3. Make it *actually attempt* one assignment. Grade it with your rubric
4. Decide per component: keep, move in-class, redesign

Two questions per component

- What is it a *proxy* for? Nobody inverts a matrix by hand, but being able to shows you understand the regression
- Is it needed *without* AI? A French essay written with AI is useless once translation is perfect

Prompt

"Read the syllabus and every assignment in this folder. For each graded component, give me a table with: (1) the competency it is meant to certify, (2) whether students need that competency without AI, (3) whether a student using a frontier model could get a top mark on it today, and (4) what you would change. Be blunt. Then complete problem set 2 yourself, as a student would, and tell me what grade it would get."

Expect the agent to score well. That is the point of the exercise.

Key takeaway. Take-home work should no longer count, Bryan wrote on the day ChatGPT launched.

Three ways to make an assignment un-cheatable, and what each one costs you.

1 In-class only

Oral exams (*viva voce*), blue-book written exams, in-class problem solving.

Works because AI is not in the room.

Cost: very time-intensive to evaluate. Hard to scale without TAs.

2 Use AI to fight AI

Students hand in a précis early. You show the class what a frontier model produces from it. The final paper must be *better than that*.

Works because the AI output becomes the floor. To beat it: archives, phone calls, site visits, own data.

Cost: you generate and read the baseline. Term papers only.

3 Un-cheatable at home

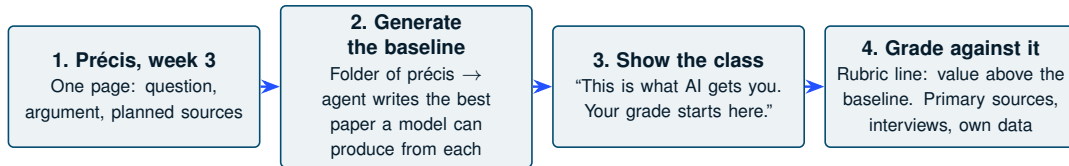
Low-stakes quizzes graded 0 or 1. Pass with 10 or more correct, *however many are wrong*. Wrong answers require the student to explain their logic.

Works because more right answers buy nothing, so cheating buys nothing. Explaining errors is where the learning happens.

Cost: question bank and automation up front. Scales to any size.

Key takeaway. Bryan runs option 3 via his product, All Day TA. The design works with any quiz tool.

The précis workflow, step by step: make the model's paper the grade floor.



Prompt for step 2

"For each précis in `precis/`, write the strongest 2,000-word term paper a frontier model can produce from it, using only what it already knows. Save as `baseline/<student>.md`. Then add a one-paragraph note on what a student would need to do to clearly beat this paper."

Practical notes

- Tell students exactly *how* to beat the baseline. Bryan's list: get into the archives, get on the phone, visit the site, create data
- Detection is a backstop, not the design. Pangram's false-positive rate is very low (Day 1), but the workflow works even if you never run it
- Require an attached note on what AI did (see the policy slide)

Rule 3: students study when tested. Regular graded checks beat study plans.

What the evidence says

- Study-tool use rises **5–10×** in exam weeks, and is far higher in courses with exams
- Study plans, reminders, and grade coaching **barely moved** performance (Oreopoulos et al.)
- An intermediate exam raised **pass rates and final grades**
- A goal for completing practice exams worked. A goal for the final grade did not

Take-home work can no longer force effort, so the checks must be frequent, graded, and hard to outsource.

How to implement it

1. Weekly quiz, pass/fail, small weight each, most weeks count
2. Generate the bank from your own notes:
"From notes/week3.md, write 15 multiple-choice questions at three levels, with distractors built from common misconceptions and a one-sentence explanation each. Output in our LMS import format."
3. Import (GIFT for Moodle, CSV for Brightspace). Randomise question order and selection
4. Verify every answer key yourself before it goes live

Key takeaway. The quiz is not the assessment. It is what makes the studying happen before it.

Rules 5–7 rest on trial evidence: AI tutoring, teacher feedback, adaptive practice.

Idea	Evidence Bryan cites	Effect
AI tutor (rule 5)	Six-week trial with Nigerian students	0.31 SD
	World Bank meta-analysis of older ed-tech	0.12 SD
	Khanmigo (NBER); Harvard physics AI tutor	Gains despite limited use
Teacher feedback (rule 6)	Feedback RCT, 1,000+ online CS teachers	+13% use of student ideas
Adaptive practice (rule 7)	Mindspark adaptive system (pre-LLM), 4 months	0.37 SD maths, 0.23 Hindi
	LLM-chosen problem sets (Taiwan), no-AI final	0.15 SD vs fixed sequence
Fetching answers from AI	Homework up, exams down (previous slides)	Does not work

Key takeaway. Old mechanisms, new cost: tutoring, mastery learning, spaced practice at scale.

Implementing rules 5–7 for free: tutor, misconception digest, levelled problem sets.

Course tutor

A shared assistant (custom GPT, Claude project, NotebookLM) loaded with *your* lecture notes and problem sets.

System prompt essentials: use the course's notation, never give the answer, ask the student to explain first, return to topics they missed.

Grounding and instructions matter more than the tool.

Misconception digest

Each week, export the questions students asked (tutor logs, forum, email), anonymised.

Prompt: *“Group these by underlying misconception, rank by frequency, and point to the lecture slide most likely to have caused each.”*

Fix the slide before the next lecture, not after the exam.

Levelled problem sets

One problem, three difficulty tiers. Assign the tier by last week's quiz result.

Prompt: *“Write this problem at three levels: scaffolded, standard, extension. Same concept, full solutions, and a rubric I can grade against.”*

The agent drafts and pre-grades. You spot-check and decide.

Key takeaway. Student questions are personal data. Anonymise before exporting and check the institutional policy once.

Rule 8: raise the bar, and write the AI policy into the syllabus.

Bryan's course policy, verbatim

"You may use AI in this course. If you use AI to write, you must specify in an attached note precisely what AI assisted with. Understand that with AI, my expectation for the quality of your ideas and writing is higher than it used to be — you need to show that you are able to provide value above and beyond AI. For readings, you must read them first yourself! Do not summarize them with AI. It will be very obvious in class if you do. I want your ideas; I can ask GPT for its thoughts without coming to class, as can your classmates."

Key takeaway. The clearer you are about where students **should** use AI and where they **must not**, the better they get the balance right.

Why raise the bar

- Randomised workplace studies: AI raised writing quality and cut completion time
- Consultants working inside AI's competence produced better work ([Dell'Acqua et al., 2026](#))
- If content-gathering is cheap, an A from ten years ago is no longer an A

Practical

- State the policy *per assignment*, not once per course
- Require the attached AI-use note and grade it
- Rubric line: sloppy citation and error-ridden prose now cost more, not less

Hands-on (15 min) — audit one of your own courses.

1. Put the syllabus and the graded assignments for one course in a folder. One past exam if you have it
2. Open the folder in VS Code and run the audit prompt from the Rules 1 and 4 slide
3. Read the table. Where do you disagree with the agent about what a component certifies?
4. Ask it to attempt the assignment you are most worried about. Grade the result with your rubric
5. Pick one component and redesign it using one of the three options: in-class, précis baseline, or threshold quiz

Debrief questions

- Which assignment surprised you most?
- Which of Bryan's eight rules is cheapest to implement in your course next semester?
- Which one would your department push back on, and why?

Slides and websites

Making slides with AI

The old path

- Open Beamer
- Write LaTeX by hand
- Fight the syntax
- Iterate slowly

Hours of mechanical work between idea and first draft.

The Claude Code path

- Open the project folder
- Hand Claude a file: *"Make a Beamer deck based on @paper.tex"*
- Compile, look at the output, iterate

This deck was built that way.

Key takeaway. This also means that we have to practice presentations in a different way

Including a style file gives Claude guidance to match the look you want.

Without a style file. Claude defaults to generic Beamer themes — bullet-heavy, blue-and-white, no action titles.

With a style file. The deck inherits your colour palette, frame structure, and title conventions.

How to build a `slides-style.md`

- Upload a few old presentations
- Ask Claude to analyse the structure, palette, and title style
- Save the result as `slides-style.md` in the project folder
- Reference it whenever you ask for a new deck

Key takeaway. The Day 1 deck, this deck, and your future decks can all share one style file. Build it once.

Live demo — transcript and code → interactive website.

The task

- Take a folder of do-files and a transcript
- Build a standalone interactive website

What Claude Code does

1. Reads the code and the transcript
2. Sees how the code constructs the figures
3. Writes a standalone HTML page — one file, no dependencies

Why this works

- Claude knows how the figure was built — it has the code
- Claude knows how to write HTML
- Putting them together is exactly where agentic AI has an advantage over chat

***Pattern:** two skills the agent already has, glued together by the folder.*

Key takeaway. Same pattern works for policy briefs, replication packages, course websites, and conference programmes.

SECTION 9

Literature

Convert files,

Zotero MCP connector

MCP servers plug the agent into external systems — know it exists, install only with a clear use case.

Model Context Protocol. A standard for letting Claude call external tools beyond the local filesystem.

Examples already in production

- GitHub — open PRs, read issues, comment on code review
- Databases — query Postgres, BigQuery, etc. directly
- Custom servers you wrote yourself

Like browser extensions, but for the agent. The list is growing fast.

Key takeaway. For most academics, today, **you do not need MCP**. Know the concept exists; install one only when you have a concrete use case.

Q & A

*Open floor. Email: claes.backman@gmail.com |
Slides, skills, guides: claesbackman.com/aarhus*

References I

Kevin A. Bryan. Eight rules for teaching in AI world. <https://kevinbryanecon.com/teachwithai.html>, August 2026.

Fabrizio Dell'Acqua, Edward McFowland III, Ethan Mollick, Hila Lifshitz, Katherine C Kellogg, Saran Rajendran, Lisa Kraye, François Candelon, and Karim R Lakhani. Navigating the jagged technological frontier: Field experimental evidence of the effects of artificial intelligence on knowledge worker productivity and quality. *Organization Science*, 37(2):403–423, 2026.

Appendix — CLAUDE.md starter template.

About me

I am a researcher in [field].

How I want you to work with me

- Ask clarifying questions before generating long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md). No generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality.
- Do not insert emoji or markdown decorations in formal docs.

Appendix — instructor's skill library.

Skill	Use
<code>/review-paper</code>	Pre-submission referee report, 8 agents
<code>/review-paper-light</code>	Fast 2-agent version, $\approx$ 1 minute
<code>/review-paper-code</code>	Paper + code alignment review
<code>/review-grant</code>	Pre-submission grant review, 6 agents
<code>/review-pap</code>	Pre-analysis-plan review
<code>/audit-analysis</code>	Adversarial audit of changed analysis code
<code>/explain-diff</code>	Explain a code change as an HTML page with a quiz
<code>/paper-version</code>	Paper $\rightarrow$ policy brief / 1-page / 5-page
<code>/pdf-to-markdown</code>	Convert PDFs to readable markdown

All at github.com/claesbackman/Al-research-feedback (MIT), linked from claesbackman.com/aarhus.
Fork them, modify them, make them yours.

Appendix — file format quick reference.

Format	Handling
.md, .tex, .txt, .bib	Native — no conversion
.py, .R, .do, .ipynb	Native — read, edit, run
.csv	Native up to a few thousand rows
.docx	Convert: <code>pandoc file.docx -o file.md</code>
.xlsx	Export to CSV first
.dta, .rds	Script-based conversion to CSV
PDF (born-digital, short)	Native; expect table mangling
PDF (long, scanned, equations)	Convert to markdown first

Appendix — signs of session drift.

- Claude ignores CLAUDE.md
- Repeats mistakes you already corrected
- Invents files that do not exist
- Gives circular answers
- Forgets the last three turns

Fix. Start a fresh session or close and reopen. Re-state the task. Re-attach the relevant file.

30+ turns or the task has shifted → start fresh. Always.